Chapter 3
Formatted Input/Output

CSE 4107 : Structured Programming I
REYNEP

3.1. The `printf()` function

```c
int i = 10;
int j = 20;
float x = 43.2892f;
float y = 5527.0f;

printf("hello, world!\n");
printf("i = %d, j = %i, x = %f, y = %f\n", i, j, x, y);
```

```
// output
hello, world!
i = 10, j = 20, x = 43.289200, y = 5527.000000
```

"String" & "conversion specification"

- `"hello, world!\n"`
  - *This is called a* `String`
  - *Whatever is inside* `"` *Double-Quotes, is always* `String`
  - `\n` - **newline** - *(ASCII Character 10)*
    - *Without this, 2nd line would be printed right beside* `hello, world!`
- `%d` - *decimal*
- `%i` - *integer*
- `%f` - *float*
  - *These are called* **"conversion specification"**
  - *They start with a* `%`
  - *They must stay inside* **"String"**
  - *They get* **"filled in"** *with values of* `i`, `j`, `x`, `y` *during printing* 🤷‍♀️

"format string"

- *The* `printf()` *function takes a* `string` *as input*
  - `printf()` *is designed to display the contents of this input* `string`
  - *now, because* `printf()` *has built-in functionality to handle* **"conversion specification"** *(* `%d`, `%` *stuffs)*
    - *this* `string` *gets a special name*
    - *this* `string` *is called* **"format string"**

- *2 kinds of stuff that can be inside this* ***"format string"***
  i. **"ordinary characters"** ➡️ *letters/buttons of a keyboard*
    - *Search* **"ASCII"** *on google*
    - `\n` *is also another* **ASCII** *character*
  ii. **"conversion specifications"** ➡️ `%d`, `%` *stuffs*
    - *by using* `%d`, `%` *stuffs, we can insert values @ specified points in the* `string` *& those values will be printed*

using `%d` / `%` stuffs

```c
int x = 50;
printf("welcome to cs%d", x);
printf("\n");
```

- *we used 1* `%d` *, so we passed in 1 integer right after the* `string`

```c
int x = 10;
int y = 20;
printf("x = %i, y = %i", x, y);
printf("\n");
```

- *we used 2* `%d` *, so we passed in 2 integer right after the* `string`
  - *so, like, formally,*
    - *the values of* `x` *,* `y` *are gonna be inserted into the* `format string` *,* **"left-to-right"** *@ the places of* **"conversion specifications"**

- *There's* ***no limit*** *on the* ***number of values*** *that can be printed by a* ***single call*** *of* `printf()`

ivan's lines

- `printf()` *must be supplied with the* `format string` *,*
  followed by any values that are to be inserted into the `string` *during printing:*

  ```c
  printf(string, expr1, expr2, …);
  ```

- *The* ***"format string"*** *may contain both*
  - *ordinary characters and*
  - **"conversion specifications"**, *which begin with the* `%` *character.*
- *A conversion specification is a placeholder representing a value to be filled in during printing*
  - `%d` *is used for* `int` *values*
  - `%i` *is used for* `int` *values*
  - `%f` *is used for* `float` *values*

- *Then he shows the code @ the start of this page, & typed beside*
  - *Ordinary characters in a* `format string` *are printed as they appear in the* `string` *;*
  - **"conversion specifications"** *are replaced*

Possible Errors

1.
```
printf("%d %d\n", i);           /*** Too many conversion specifications ***/
```
```
outputs:
    50 1561391600
    50 <some-random-int>
```

- **Compilers** *aren't required to check that the number of* **"conversion specifications"** *in a* **"format string"** *matches the number of output items.*

2.
```
printf("%d\n", i, j);           /*** Too few  conversion specifications ***/
```
```
outputs:
    50
```

- *works,* `j` *is just ignored.*

3.
```
int k = 50;
float y = 3.14;
printf("%f %d\n", k, y);        /***   wrong  conversion specifications ***/
```
```
outputs:
    0.000000 1610612736
```

- `float` *is stored way differently than* `int`
  - *so, when a really small* `float` *like* `3.14` *is* **"interpreted"** *as* `int`*, smth like* `1610612736` *comes out!*
    - *same goes for the other way around.*
- *If the programmer uses an incorrect specification, the program will produce meaningless output, like in this case.*
- **Compilers** *aren't required to check that a conversion specification is appropriate*

3.1.1. "Conversion Specifications" / "Format Specifiers"

- *Ivan also calls these **"Conversion Specifications"**, by **"Format Specifiers"***
  - *iguess, because they specify stuff inside* **"Format Strings"**
  - *actually, the whole world calls these* `%` *stuffs* **"Format Specifiers"**

%ক

- ***Numbers***

`%ক`	***Bits***	***Description***	***Range / Notes***	`%llক`	***Bits***	***Range / Notes***
`%i`	*32-bit*	`integer`	*max: $2^{31} - 1$*	`%lli`	*64-bit*	*max: $2^{63} - 1$*
`%u`	*32-bit*	***Unsigned*** `int`	*Positive or zero*	`%llu`	*64-bit*	
`%d`	*32-bit*	***Decimal*** `int` *- Same as* `%i`		`%lld`	*64-bit*	
`%x`	*32-bit*	***Hexadecimal*** *(Lower-case)*		`%llx`	*64-bit*	
`%X`	*32-bit*	***Hexadecimal*** *(Upper-case)*		`%llX`	*64-bit*	
`%o`	*32-bit*	***Octal***		`%llo`	*64-bit*	
`%f`	*32-bit*	`float`	*max ≈ 3.4×10^{38}*	`%lf`	*64-bit*	*max ≈ 1.8×10^{308}*
`%a`	*32-bit*	***Hex*** `float`	*C99 style*	`%la`	*64-bit*	

  - *we &* `ivan` *will only use* `%i` , `%d` & `%f`
  - `64-bit`
    - *integer :* `%lli` , `%lld` , `%llu`
    - *float :* `%lf`
  - `16-bit`
    - `%hi` , `%hd` , `%hu`

- ***Scientific Notation***

`%ক`	***Bits***	***Description***
`%e`	*64-bit*	*Scientific (Lower-case)*
`%E`	*64-bit*	*Scientific (Upper-case)*
`%g`	*64-bit*	*Shortest of* `%f` *or* `%e`

Char, Letters, Strings

`%ক`	***Bits***	***Description***	***Range / Notes***
`%c`	*8-bit*	`char`	*0–127 (ASCII)*
`%s`		`char[] str`	
`%p`	*64-bit*	`pointer`	

- ***Tricks*** *- that we won't be using / that might confuse us*

Specifier	***Description***
`%#x`	*Hex with* `0x` *prefix*
`%+d`	*Show sign for positive*
`%05d`	*Zero-padded width 5*

  - `%n` *- Saves The Number of Characters Printed so far by current call of printf() into variable*

```c
int k = 40; float x = 839.21f;

printf("|%05d|\n", k);
printf("|%+d|\n", k);
```

```
// outputs

|00040|
|+40|
```

3.1.2. Gaps

```c
int k = 40;
printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
```

```
// outputs: using '·' to represent '<space>'
|40|···40|40···|··040|040··|
```

- `%d` - `int` *using minimum amount of space*
- `%5d` - `int` *using minimum of 5 characters.* ***"Right-Justified"*** *[1]*
- `%-5d` - `int` *using minimum of 5 characters.* ***"Left-Justified"***
- `%5.3d` - `int` *using minimum of 5 characters.* ***"Right-Justified"****. At Least 3 Digits. [2]*
- `%-5.3d` - `int` *using minimum of 5 characters.* ***"Left-Justified"****. At Least 3 Digits.*

- *note-1*
  - *Since variable* `k` *requires only two characters* `40` , **three spaces** *were added*
- *note-2*
  - *This one is actually confusing.*
  - *So, do some testing if you need.*
  - *Adds extra* **"Leading Zeroes"** *if needed*

```c
float x = 839.21f;
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// using '·' to represent '<space>'
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

- `%f` - `float` *using minimum amount of space, to show the* ***"actual value"***
  - *for our case it shows* `839.210022`
  - *cz binary can't store* `839.21` *exactly. The closest it can store is* `839.210022`
  - *This approximation is called* **"floating point error margin"**

- `%10f` - `float` *using minimum of 10 characters.*
- `%-12f` - `float` *using minimum of 10 characters.* ***"Left-Justified"***
- `%10.3f` - `float` *using minimum of 10 characters.* ***"Right-Justified"****. At least 3 digits* ***"after*** `.` ***"***
- `%10.3f` - `float` *6chars* `.` *3digits ---------- [6 + 3 + 1 for* `.` *= 10]*
- `%10.3e` - `float` *2chars* `.` *3digits* `e` ± `02`
- `%10.3e` - `float` *2chars* `.` *3digits* `e` *3chars*
- `%15.3e` - `float` *7chars* `.` *3digits* `e` *3chars*
- `%15.7e` - `float` *3chars* `.` *7digits* `e` *3chars*

- `%-10g` - `float` *chooses between* `%f` *&* `%e` *. Omits* ***"Trailing/Leading Zeroes"***
  - *For our example case, chooses* `%f`
  - *Acts exactly like* `%-10.2f`

3.1.3. Gaps - Summaries

- `%5d` *v/s* `%10f`
  - `%5d` : `int` *using min. 5 chars.* **"Right-Justified"**
  - `%10f` : `float` *using min. 10 chars.* **"Right-Justified"**. `.` *is an extra char which will always be there.*
  - `%10e` : `exponential` *using min. 10 chars.* **"Right-Justified"**. `e+00` *are 4 extra char.*

- `%5.3d` *v/s* `%10.3f`
  - `%5.3d` : `int` *using min. 5 chars.* **"Right-Justified"**. *At Least 3 Digits* **"using** `0` **"**
  - `%10.3f` : `float` *using min. 10 chars.* **"Right-Justified"**. *At least 3 digits* **"after** `.` **"**

3.1.4. `___` $\%-n_1.n_2$क

ex.	generalized
`%i` , `%d` , `%u` , `%f`	%क
`%lli` , `%lld` , `%lf`	% *ll* क
`%5d` , `%15f`	% n_1क
`%5.3d` , `%10.3f`	% n_1 . n_2क
`%-5.3d` , `-%10.3f`	% − n_1 . n_2क

- *ll* : *64-bit stuffs*
- − : ***"Left-Justified"***
- n_1 : *using minimum of* n_1 *chars. (inclusing* `.` *-decimal-point, or* `e+00` *)*
- n_2 : *at least* n_2 *digits*
  - `int` *at least* n_2 *digits* **"using `0` "**
  - `float` *at least* n_2 *digits* **"after `.` "**

```c
int k = 40;
float x = 839.21f;

printf("|%d|%5d|%-5d|%5.3d|-%5.3d|\n", k, k, k, k, k);
printf("|%f|\n", x);
printf("|%10f|\n", x);
printf("|%10.3f|%10.3e|%-10g|%-12f\n", x, x, x, x);
```

```
// outputs: using '·' to represent '<space>'


|40|···40|40···|··040|040··|
|839.210022|
|839.210022|
|···839.210|·8.392e+02|839.21···|839.210022··|
```

ivan's lines (edited) 🟩

- *A conversion specification can have the form* $\%n_1.n_2$क *or* $\%-n_1.n_2$क ,
  - क *is a letter.*
  - n_1 *and* n_2 *are integer* **constants**
  - n_1 *and* n_2 *are optional*
    - *if* n_2 *is omitted then, the* `.` *dot that separates* n_1 *and* n_2 *is also dropped*

- `%10.2f`
  - n_1 *is* `10`
  - n_2 *is* `2`
  - क *is* `f`
- `%10f`
  - n_1 *is* `10`
  - n_2 *is missing (so is the* `.` *)*
  - क *is* `f`
- `%.2f`
  - n_1 *is missing*
  - n_2 *is* `2`
  - क *is* `f`

- $\%n_1.n_2$क $\implies$ ***"format specifier"*** $\implies$ $\%m.pX$
  - $n_1 \to m$ $\implies$ **"minimum field width"**
  - $n_2 \to p$ $\implies$ **"precision"**
  - क $\to X$ $\implies$ **"conversion specification"**
    - *I hate ivan's decision of using 'X' capital-x here, because,* `%X` *actually has it's own actual meaning "***Hexadecimal (Upper-Case)***"*

ivan's lines : (unedited) - details

- *Slide 6*
  - *A* **"conversion specification"** *can have the form* `%m.pX` *or* `%-m.pX`*, where* `m` *and* `p are integer constants and X` *is a letter*
  - *Both* `m` *and* `p` *are optional; if* `p` *is omitted, the period that separates* `m` *and p is also dropped*
  - *In the conversion specification* `%10.2f`*,* `m` *is* `10`*,* `p` *is* `2`*, and* `X` *is* `f`
  - *In the specification* `%10f`*,* `m` *is* `10` *and* `p` *(along with the* `.` *period) is missing,*
    - *but in the specification* `%.2f`*,* `p` *is* `2` *and* `m` *is missing*

- *Slide 7+8 :* $n_1 \rightarrow m$
  - *The* **"minimum field width"**, `m`*, specifies the minimum number of characters to print*
  - *If the value to be printed requires* **fewer than** m *characters, it is* **"left/right-justified"** *within the field.*
    - `%5d` - **"Right-Justified"**
    - `%-5d` - **"Left-Justified"**
    - `%4d` *displays the number* `123` *as* `•123`*.*
      - `•` *represents the space character*
  - *If the value to be printed requires* **more than** m *characters, the field width automatically expands to the necessary size*
    - *don't get confused*
    - *this simply means, in this case,*
      - `%5d` *for* `123456` *the effect is same as* `%d`
  - `%-5d` *Putting a minus sign in front of* m *causes* **"left justification"**
    - *The specification* `%-4d` *would display* `123` *as* `123•`

- *Slide 9 :* $n_2 \rightarrow p$
  - *The meaning of the* **"precision"**, p*, depends on the choice of* X*, the* **"conversion specifier"**
  - `%.3d` *- The* `d` *specifier is used to display an integer in decimal form.*
    - p *indicates the* **"minimum number of digits"** *to display (extra zeros are added to the beginning of the number if necessary).*
    - *If* p *is omitted, it is assumed to be 1*

- *Slide 10 :* $n_2 \rightarrow p$
  - `%e` *- Exponential format*
    - p *indicates how many digits should appear after the decimal point (the default is* `6` *including* `e+00` *???). If* p *is 0, no decimal point is displayed.*
  - `%f` *- Float - Non-Exponential Format (ivan calls 'non-exponential' = 'fixed decimal')*
    - p *has the same meaning as above*
  - `%d` *- Integer*
    - *Minimum number (*p*) of digits to display (append extra zeros at the beginning if needed)*

- *Slide 11*
  - *Show the use of format specifiers for both* `int` *and* `float`
    - *Use* `%d`*,* `%5d`*,* `%-5d`*,* `%5.3d` *for* `int k = 40`
    - *Use* `%10.3f`*,* `%10.3e` *for* `float y = 839.21`

3.1.5. Escape Sequence

```
printf("Item\tUnit\tPurchase\n\tPrice\tDate\n");
```

```
Item    Unit    Purchase
        Price   Date
```

- *The `\n` code that we used in format strings since day 1, is called an **"escape sequence"***
- *Control Characters:*
  - *Alert (Bell) : `\a` - printing `\a` causes an audible beep on most machines. (well, REYNEP never saw it. 🥹)*
  - *NewLine : `\n` - printing `\n` advances the cursor to the beginning of the next line.*
  - *Horizontal Tab : `\t` - printing `\t` moves the cursor to the next tab stop*
  - *BackSpace : `\b` - printing `\b` moves the cursor back one position*

- *Special Characters:*
  - *Backslash : `\\` - `printf(" ---- \\ ---- ");`*
  - *Double Qotes : `\"` - `printf(" ---- \" ---- ");`*

3.2. the `scanf()` function

```
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

```
1 -20 .3 -4.0e3
    // Say, the user writes this line and hits enter
    // Then
            m = 1       x = 0.3
            n = -20     y = -4000.0
```



3.2.1. ivans lines

Just as `printf()` prints output in a specified format, `scanf()` reads input according to a particular format.

A `scanf()` format string, like a `printf()` format string, may contain both ordinary characters and conversion specifications. The conversions allowed with scanf are essentially the same as those used with printf.

In many cases, a `scanf()` format string will contain only these conversion specifications, as in the following example:

```
int m, n;
float x, y;
scanf("%d %d %f %f", &m, &n, &x, &y);
```

Suppose that the user enters the following input line:

```
1 -20 .3 -4.0e3
```

and then presses enter.

`scanf()` will read the line, converting its characters to the numbers they represent, and then assign `1`, `-20`, `0.3`, and `-4000.0` to `m`, `n`, `x`, and `y`, respectively.

Note that,

```
scanf("%d %d %f %f", &m, &n, &x, &y);
scanf("%d%d%f%f", &m, &n, &x, &y);
```

both would have had the same effect. “Tightly packed” format strings like `"%d%d%f%f"` are common in `scanf()` calls.

3.2.2. `scanf()` traps (unedited)

`scanf()`, like `printf()`, contains several traps for the unwary.

- *When using* `scanf()`*, the programmer must check that,*
  - **the number of** *conversion specifications matches* **the number of** *input variables*
  - *and that each conversion is appropriate for the corresponding variable*
    - *as with* `printf()`*, the compiler isn't required to check for a possible mismatch.*
- *Another trap involves the* `&` *symbol, which normally precedes each variable in a* `scanf()` *call.*
- *The* `&` *is usually (but not always) required, and it's the programmer's responsibility to remember to use it.*

3.2.3. How `scanf()` works 🟩 (unedited)

`scanf()` can actually do much more than I've indicated so far. It is essentially a "pattern-matching" function that tries to match up groups of input characters with conversion specifications.

Like the `printf()` function, `scanf()` is controlled by the format string. When it is called, `scanf()` begins processing the information in the string, starting at the left. For each conversion specification in the format string, `scanf()` tries to locate an item of the appropriate type in the input data, skipping blank space if necessary. `scanf()` then reads the item, stopping when it encounters a character that can't possibly belong to the item. If the item was read successfully, `scanf()` continues processing the rest of the format string. If any item is not read successfully, `scanf()` returns immediately without looking at the rest of the format string (or the remaining input data).

As it searches for the beginning of a number, `scanf()` ignores **white-space characters** (the space, horizontal and vertical tab, form-feed, and new-line characters).

As a result, numbers can be put on a single line or spread out over several lines. Consider the following call of `scanf()`:

```
scanf("%d%d%f%f", &i, &j, &x, &y);
```

Suppose that the user enters three lines of input:

```
  1
-20   .3
   -4.0e3
```

scanf sees one continuous stream of characters:

```
••1¤-20•••.3¤•••-4.0e3¤
```

(I'm using • to represent the space character and ¤ to represent the new-line character.)

```
••1¤-20•••.3¤•••-4.0e3¤
ssrsrrrsssrrsssrrrrrr
```

`s` under a character indicates that it was skipped
`r` indicates it was read as part of an input item

> `scanf()` "peeks" at the final new-line character without actually reading it. This **new-line will be the first character read by the next call** of `scanf()`.

3.2.4. How `scanf()` works - II - 🟩 (unedited)

What rules does scanf follow to recognize an integer or a floating-point number?

- *When asked to read an integer*
  - *scanf first searches for a digit, a plus sign, or a minus sign;*
  - *it then reads digits until it reaches a nondigit.*

- *When asked to read a floating-point number scanf looks for*
  - *a plus or minus sign (optional), followed by*
  - *a series of digits (possibly containing a decimal point), followed by*
  - *an exponent (optional).*
    - *An exponent consists of the letter e (or E), an optional sign, and one or more digits.*

The `%e`, `%f`, and `%g` conversions are interchangeable when used with `scanf()`; all three follow the same rules for recognizing a floating-point number.

3.2.5. How `scanf()` works - III : Demonstration

When scanf encounters a character that can't be part of the current item, the character is "put back" to be read again during the scanning of the next input item or during the next call of scanf. Consider the following (admittedly pathological) arrangement of our four numbers:

```
INPUT: 1-20.3-4.0e3¤

scanf("%d%d%f%f", &m, &n, &x, &y);
    // Let's use the same call of scanf as before
```

Here's how scanf would process the new input:

1. `%d` - *The first nonblank input character is* `1` *; since integers can begin with* `1` *, scanf then reads the next character,* `-` *. Recognizing that* `-` *can't appear inside an integer,* `scanf()` *stores* `1` *into* `m` *and puts the* `-` *character back.*
    - *for it shall be processed by the next call of* `scanf()` *or the next* `%` *item.*

2. `%d` - *scanf then reads the characters* `-` *,* `2` *,* `0` *, and* `.` *(period). Since an integer can't contain a decimal point, scanf stores* `-20` *into* `m` *and puts the* `.` *character back.*

3. `%f` - *scanf reads the characters* `.` *,* `3` *, and* `-` *. Since a floating-point number can't contain a minus sign after a digit, scanf stores* `0.3` *into* `x` *and puts the - character back.*

4. `%f` - *Lastly, scanf reads the characters* `-` *,* `4` *,* `.` *,* `0` *,* `e` *,* `3` *, and* `¤` *(new-line). Since a floating-point number can't contain a new-line character, scanf stores* -4.0×10^3 *into* `y` *and puts the new-line character back.*
    - **NOTE: this `¤` (new-line) will be read by the next `scanf()`**

> In this example, scanf was able to match every conversion specification in the format string with an input item.
> **Since the new-line character wasn't read, it will be left for the next call of scanf.**

3.2.6. Ordinary Characters in Format Strings 🟩 (unedited)

When scanf encounters a character that can't be part of the current item, the character is "put back" to be read again during the scanning of the next input item or during the next call of scanf.

The concept of pattern-matching can be taken one step further by writing format strings that contain ordinary characters in addition to conversion specifications. The action that scanf takes when it processes an ordinary character in a format string depends on whether or not it's a white-space character.

- ***White-space characters:*** *When it encounters one or more white-space characters in a format string, scanf reads white-space characters from the input until it reaches a non-white-space character (which is "put back")*

- ***Other characters:*** *When it encounters a non-white-space character in a format string, scanf compares it with the next input character*
    - *If they match, scanf discards the input character and continues processing the format string*
    - *If they don't match, scanf puts the offending character back into the input, then aborts*

- *Examples:*

```
scanf("%d/%d", &x, &y)

input: •5/•96
    x=5, y=96
```

If the input is •5•/•96 , scanf fails, because the / in the format string doesn't match the space in the input

3.2.7. Confusing `printf()` with `scanf()` (unedited)

- *Don't use ampersand in printf*
  - `printf("%d %d\n", &i,&j);` - *Wrong*
- *Don't use ordinary characters in scanf unless it is necessary*
  - `scanf("%d, %d", &i, &j); // Not error, but be careful`
- *Putting a new-line character at the end of a scanf format string is usually a bad idea*
- *If the format string is* `"%d\n"`
  - *scanf will*
    - *skin white-space*
    - *read an integer*
    - *then skip to next non-white-space character*
    - *thus -> cause interactive programs to hang*
- *Consider a C program that prompts the user to enter two fractions and then displays their sum*
- *Fractions are customarily denoted as: numerator/denominator*

```
Sample program output:
Enter first fraction: 5/6
Enter second fraction: 3/4
The sum is 38/24
```